Pengembangan Sistem Self-Healing Test Automation Berbasis Large Language Model untuk Meningkatkan Reliabilitas Pengujian pada Framework Katalon dalam CI/CD Pipeline

Proposal Tugas Akhir

Kelas MK Penulisan Proposal (CCH4A3)

1302220005

Wildan Syukri Niam

Program Studi Sarjana Rekayasa Perangkat Lunak

Fakultas Informatika

Universitas Telkom

Bandung

2025

Lembar Persetujuan

Pengembangan Sistem Self-Healing Test Automation Berbasis Large Language Model untuk Meningkatkan Reliabilitas Pengujian pada Framework Katalon dalam CI/CD Pipeline

Development of LLM-Based Self-Healing Test Automation System to Improve Test Reliability on Katalon Framework in CI/CD Pipeline

NIM : 1302220005

Wildan Syukri Niam

Proposal ini diajukan sebagai usulan pembuatan tugas akhir pada
Program Studi Sarjana Rekayasa Perangkat Lunak
Fakultas Informatika Universitas Telkom

Bandung, 8 Desember 2025

Menyetujui

Calon Pembimbing 1	Calon Pembimbing 2
Dr. Eng. Jati Hiliamsyah Husen, S.T., M.Eng.	Irham Hafidz Masyfi, S.Kom
NIP: 20920040	NIP: 211963

ABSTRAK

Fragilitas locator merupakan salah satu penyebab utama kegagalan pengujian antarmuka pengguna (UI) dalam proses otomasi, terutama ketika perubahan kecil pada struktur Document Object Model (DOM) membuat skrip uji tidak lagi mampu menemukan elemen target. Kondisi ini memicu false failures, meningkatkan beban pemeliharaan, dan mengganggu stabilitas pipeline CI/CD. Berbagai mekanisme self-healing konvensional telah digunakan untuk mengurangi masalah tersebut, namun sebagian besar masih mengandalkan pendekatan berbasis aturan yang terbatas dalam memahami perubahan konteks UI. Sejalan dengan kemajuan teknologi Large Language Model (LLM), muncul peluang untuk memanfaatkan kemampuan reasoning model dalam menghasilkan perbaikan locator yang lebih adaptif dan kontekstual.

Penelitian ini bertujuan mengembangkan dan mengevaluasi mekanisme self-healing berbasis LLM yang terintegrasi dengan workflow Katalon Studio dan Jenkins. Sistem yang diusulkan mencakup modul deteksi kegagalan locator, analisis perubahan DOM melalui DOM differencing, generasi locator baru menggunakan model LLM, serta verification pipeline untuk memastikan keakuratan perbaikan sebelum diterapkan pada test suite. Evaluasi dilakukan melalui eksperimen pada test suite UI dengan mensimulasikan perubahan DOM terhadap sekitar seratus test case untuk menilai keberhasilan perbaikan, efisiensi waktu, dan dampaknya terhadap reliabilitas pengujian.

Hasil penelitian ini diharapkan dapat menunjukkan kemampuan LLM dalam menghasilkan perbaikan locator secara otomatis dengan tingkat keberhasilan yang kompetitif dibandingkan pendekatan konvensional, sekaligus memberikan kontribusi terhadap peningkatan stabilitas dan pengurangan beban maintenance dalam pipeline pengujian modern.

Kata kunci: CI/CD, Katalon Studio, Large Language Model, Locator UI, Self-Healing, Test Automation

1. PENDAHULUAN

1.1. Latar Belakang

Otomasi pengujian antarmuka pengguna (*User Interface* atau UI) telah menjadi komponen krusial dalam menjamin kualitas perangkat lunak modern. Namun, implementasinya masih menghadapi tantangan persisten terkait *fragilitas locator* [1]. Dalam banyak kasus, perubahan minor pada UI eperti modifikasi atribut elemen, perubahan *ID*, atau pergeseran struktur *Document Object Model* (DOM) dapat menyebabkan skrip uji gagal menemukan elemen target, meskipun fungsionalitas aplikasi sebenarnya berjalan normal [1]. Kondisi ini memicu tingginya angka *false failures*, terutama ketika pengujian dijalankan dalam *pipeline Continuous Integration/Continuous Delivery* (CI/CD), di mana satu kegagalan dapat menghentikan keseluruhan proses *build* dan membebani tim *Quality Assurance* (QA) dengan proses *debugging* yang repetitif [2].

Masalah utama yang terjadi saat ini adalah keterbatasan mekanisme pemulihan (*self-healing*) konvensional. Sebagian besar *framework* otomasi industri, termasuk Katalon Studio, umumnya mengandalkan pendekatan berbasis aturan (*rule-based*) atau heuristik sederhana untuk menangani kegagalan *locator*. Mekanisme ini bekerja dengan mencari atribut cadangan statis (seperti name atau class) ketika *locator* utama gagal. Meskipun efektif untuk perubahan ringan, pendekatan ini sering kali gagal ketika terjadi perubahan struktur DOM yang kompleks atau ketika atribut semantik elemen berubah total. Studi menunjukkan bahwa pendekatan tradisional ini kurang mampu menangkap konteks semantik dari elemen UI, sehingga solusi yang ditawarkan sering kali tidak akurat atau memerlukan intervensi manual yang intensif [1], [3]. Akibatnya, proses *test automation* sering menjadi *bottleneck* karena tingginya ketergantungan pada pemeliharaan skrip yang rapuh [4].

Pendekatan *TestOps* menekankan bahwa pengujian harus didukung oleh *workflow* yang adaptif dan mampu bertahan terhadap evolusi aplikasi [2]. Oleh karena itu, diperlukan solusi yang tidak hanya "mencari atribut pengganti", tetapi mampu

"memahami" perubahan antarmuka layaknya penguji manusia. Kemajuan teknologi *Large Language Model* (LLM) menawarkan peluang baru untuk mengatasi keterbatasan ini melalui kemampuan *reasoning* (penalaran) dan pemahaman semantik yang mendalam [3].

Berbeda dengan metode konvensional, LLM memiliki potensi untuk menganalisis potongan kode DOM dan memahami fungsi elemen berdasarkan konteks sekitarnya, bukan hanya pencocokan *string* yang kaku. Perusahaan teknologi besar seperti Meta dan Ericsson telah mulai mengeksplorasi penggunaan LLM untuk mendukung pemeliharaan tes otomatis [5],[6]. Namun, penelitian yang secara spesifik mengintegrasikan kemampuan *self-healing* berbasis LLM ke dalam *workflow* CI/CD dengan menggunakan *tools* industri standar seperti Katalon Studio masih sangat terbatas. Penelitian ini bertujuan untuk mengisi celah tersebut dengan mengembangkan sistem *self-healing* adaptif yang memanfaatkan LLM untuk memperbaiki *locator* secara otomatis, sehingga dapat meningkatkan reliabilitas dan efisiensi pengujian dalam ekosistem pengembangan perangkat lunak modern [1], [3].

1.2. Perumusan Masalah

Rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana tingkat keberhasilan perbaikan locator yang dihasilkan oleh LLM dibandingkan dengan pendekatan self-healing konvensional yang ada saat ini?
2. Apa dampak penerapan LLM-based self-healing terhadap efisiensi waktu dan reliabilitas test suite dalam pipeline pengujian otomatis?
3. Seberapa efektif penggunaan Large Language Model (LLM) dalam menghasilkan perbaikan otomatis terhadap locator UI yang rusak pada pengujian otomatis?

1.3. Tujuan

Tujuan dari penelitian ini adalah untuk mengevaluasi efektivitas penerapan *Large Language Model* (LLM) dalam mekanisme self-healing pada pengujian antarmuka

pengguna (UI) otomatis. Penelitian ini berfokus pada pemanfaatan LLM untuk menghasilkan perbaikan locator secara otomatis, serta menganalisis dampaknya terhadap efisiensi dan reliabilitas proses pengujian dalam konteks industri.
Secara khusus, tujuan penelitian ini adalah:

1. Mengevaluasi tingkat keberhasilan LLM dalam menghasilkan perbaikan locator UI secara otomatis, dan membandingkannya dengan pendekatan self-healing konvensional yang berbasis fallback statis.
2. Mengukur dampak integrasi LLM-based self-healing terhadap efisiensi waktu dan reliabilitas test suite, termasuk peningkatan pass rate, pengurangan waktu maintenance, dan kestabilan eksekusi dalam pipeline CI/CD.
3. Menganalisis efektivitas model LLM dalam memahami konteks DOM dan menyarankan locator yang valid secara semantik, pada berbagai variasi perubahan UI yang terjadi di lingkungan pengujian.

1.4. Rencana Kegiatan

Penelitian ini dirancang dengan pendekatan rekayasa perangkat lunak berbasis eksperimen, didukung oleh studi literatur dan pengujian kuantitatif terhadap sistem prototipe. Rangkaian kegiatan dirancang untuk menjawab rumusan masalah serta mencapai tujuan yang telah ditetapkan.

1. Kajian Pustaka

Tahap pertama penelitian adalah melakukan kajian pustaka secara mendalam untuk memahami persoalan fragilitas dalam otomasi pengujian antarmuka pengguna. Pada tahap ini, peneliti menelaah literatur terkait masalah *flaky locator*, penyebab perubahan struktur DOM yang memicu kegagalan *test*, serta tantangan yang muncul pada pipeline CI/CD. Kajian juga mencakup penelitian mengenai mekanisme *self-healing* pada berbagai *testing tools* seperti Katalon Studio untuk mengetahui solusi yang telah digunakan dalam praktik industri. Selain itu, peneliti menelusuri riset terkini mengenai pemanfaatan Large Language Models (LLM) dalam konteks otomasi testing guna mengidentifikasi potensi penerapannya pada perbaikan locator, termasuk metode verifikasi keluaran AI dan pendekatan integrasi teknologi berbasis AI ke dalam alur kerja pengujian. Tahap kajian pustaka ini

bertujuan membangun fondasi konseptual yang kuat bagi perancangan sistem yang akan dikembangkan.

2. Metode Pengumpulan Data

Tahap kedua melibatkan proses pengumpulan data melalui pendekatan eksperimental terhadap *test suite* yang dijalankan pada aplikasi OmniX sebagai objek studi. Pada tahap ini, peneliti menyiapkan sekitar 100 *test case* antarmuka pengguna yang mewakili skenario umum seperti proses login, pengisian formulir, navigasi halaman, dan validasi elemen visual. Selanjutnya, dilakukan serangkaian manipulasi terhadap struktur DOM, seperti mengubah atribut, memodifikasi posisi elemen, atau mengganti ID dan kelas, untuk mensimulasikan kondisi perubahan yang lazim terjadi pada pengembangan aplikasi. Setiap eksekusi *test suite* menghasilkan data mengenai jumlah pengujian yang gagal, keberhasilan perbaikan otomatis, akurasi locator yang dihasilkan, potensi terjadinya kesalahan perbaikan (*false healing*), serta perbandingan waktu perbaikan secara manual dan otomatis. Data inilah yang menjadi dasar bagi proses analisis dan evaluasi keberhasilan sistem.

3. Rancangan Penelitian

Tahap ketiga berfokus pada perancangan dan implementasi sistem *self-healing* yang terdiri dari beberapa komponen utama. Pada fase ini, peneliti mengembangkan modul *Failure Detector* yang mampu mengidentifikasi penyebab kegagalan locator dengan menganalisis log eksekusi dari Katalon Studio. Setelah itu, peneliti membangun modul *Locator Generator* berbasis LLM yang memanfaatkan konteks DOM dan informasi *test object* untuk menghasilkan alternatif locator yang lebih stabil. Keluaran dari modul ini kemudian diuji melalui *Verification Module* yang memastikan bahwa locator yang dihasilkan valid secara sintaks, dapat dieksekusi pada DOM aplikasi, dan tetap stabil ketika diuji ulang. Seluruh komponen tersebut diintegrasikan ke dalam lingkungan otomasi pengujian menggunakan Katalon Studio dan Jenkins, sehingga *healing engine* dapat berfungsi sebagai modul eksternal yang berinteraksi dengan repositori objek uji dan hasil eksekusi. Tahap

ini memastikan bahwa prototipe sistem dapat berjalan secara end-to-end dalam alur pengujian yang sesungguhnya.

4. Evaluasi dan Penyimpulan

Tahap terakhir penelitian melibatkan pelaksanaan eksperimen untuk membandingkan performa *test suite* sebelum dan sesudah penerapan mekanisme *self-healing* berbasis LLM. Pada tahap ini, peneliti menjalankan dua skenario pengujian, yaitu baseline tanpa *self-healing* dan skenario eksperimen dengan sistem aktif. Hasil dari kedua skenario dianalisis menggunakan pendekatan kuantitatif deskriptif untuk menilai sejauh mana mekanisme *self-healing* mampu meningkatkan stabilitas dan keberhasilan eksekusi. Analisis difokuskan pada peningkatan *pass rate*, peningkatan tingkat keberhasilan perbaikan locator, pengurangan waktu yang dibutuhkan untuk *maintenance*, serta identifikasi jumlah *false healing* yang muncul. Hasil eksperimen kemudian divisualisasikan melalui grafik, tabel, dan dokumentasi log untuk memudahkan interpretasi. Berdasarkan keseluruhan temuan, peneliti menarik kesimpulan mengenai efektivitas LLM dalam melakukan perbaikan locator secara otomatis dan keberlanjutannya untuk diterapkan pada workflow pengujian perangkat lunak di lingkungan industri.

1.5. Jadwal Kegiatan

Jadwal pelaksanaan dibuat berdasarkan rencana kegiatan. Bar-chart bisa dibuat per bulan atau per minggu. Contoh bar-chart:

Kegiatan	**Bulan**					
	1	2	3	4	5	6
Kajian Pustaka dan Pengumpulan Data						
Perancangan Arsitektur & Sistem						
Validasi dan Evaluasi Rancangan						
Penyusunan Laporan Akhir dan Presentasi						

*Keterangan: shading warna *grayscale*

2. KAJIAN PUSTAKA

2.1. Studi Terkait

Pemanfaatan Large Language Model (LLM) untuk berbagai tugas dalam software testing berkembang cepat. Survei menyeluruh oleh Wang et al. [3] mengkaji aplikasi LLM pada kegiatan seperti *test generation*, *oracle creation*, *debugging*, dan *automated repair*, namun menegaskan bahwa sebagian besar penelitian masih berfokus pada fase pembuatan test dan perbaikan tingkat kode, sedangkan aspek pemeliharaan test suite, khususnya untuk UI testing yang rentan terhadap perubahan struktur DOM masih kurang mendapat perhatian. Studi terkait pada ranah unit testing, misalnya yang mengarahkan LLM berdasarkan *test intention [7]* atau menggunakan *method slicing* untuk cakupan tinggi [8], memperlihatkan kapabilitas reasoning model dalam konteks kode, tetapi mereka tidak secara langsung menyelesaikan problem fragilitas locator pada pengujian antarmuka.

Di ranah industri, bukti awal penerapan LLM untuk mendukung perbaikan dan pemeliharaan test mulai muncul. Upaya-upaya yang dilaporkan menunjukkan bahwa LLM dapat menjadi komponen yang membantu dalam proses pemeliharaan otomatis, memberikan rekomendasi atau saran perbaikan yang dipadukan dengan mekanisme verifikasi sebelum diadopsi ke dalam test suite [5], [6]. Temuan industri ini mengindikasikan potensi teknologi LLM untuk meningkatkan efektivitas maintenance, namun adaptasinya terhadap masalah khusus UI locator dan integrasinya ke workflow CI/CD masih belum dipetakan secara komprehensif.

Masalah pemeliharaan pengujian juga perlu dipandang dalam konteks proses pengembangan modern. Praktik DevOps dan gagasan TestOps menempatkan pengujian sebagai bagian integral dari workflow rilis yang menuntut otomatisasi, kolaborasi lintas-peran, dan stabilitas pipeline [2], [4], [9]. Dalam konteks tersebut, beban pemeliharaan skrip uji yang rapuh akan berdampak tidak hanya pada aspek teknis tetapi juga pada efektivitas operasional tim dan ritme rilis, sehingga memunculkan kebutuhan akan mekanisme adaptif yang dapat bekerja dalam alur CI/CD.

Dukungan teknis terhadap kebutuhan adaptif tersebut tercermin pada sejumlah studi yang membahas tantangan stabilitas test dan teknik mitigasinya. Pendekatan neurosymbolic untuk memperbaiki *flaky tests* [10], serta metode *similarity-based* untuk melokalisasi elemen web secara lebih robust [1], menunjukkan bahwa kombinasi teknik heuristik/similarity dan reasoning terstruktur dapat membantu mengurangi ketergantungan pada locator yang rapuh. Selain itu, penelitian empiris yang menyoroti peran faktor infrastruktur dan portabilitas flakiness lintas bahasa pemrograman [11], [12] menggarisbawahi bahwa solusi pemeliharaan harus mempertimbangkan variabilitas lingkungan eksekusi selain perubahan DOM itu sendiri.

Meskipun berbagai studi menunjukkan potensi LLM dalam menghasilkan maupun memperbaiki test, serta terdapat upaya awal untuk mendukung proses maintenance di lingkungan industri, penelitian yang secara khusus menargetkan mekanisme *self-healing* untuk UI locator masih sangat terbatas. Tidak ditemukan pendekatan yang

mengombinasikan analisis perubahan DOM secara terstruktur dengan pemanfaatan LLM untuk menghasilkan rekomendasi locator yang relevan secara kontekstual, sekaligus mengintegrasikan proses tersebut ke dalam workflow CI/CD melalui lapisan verifikasi otomatis. Dengan demikian, terdapat kebutuhan akan model pemeliharaan berbasis LLM yang tidak hanya mampu memahami perubahan antarmuka secara semantik, tetapi juga dapat beroperasi secara aman, terukur, dan kompatibel dengan pipeline industri modern. Penelitian ini berupaya mengisi celah tersebut melalui perancangan dan evaluasi mekanisme self-healing berbasis LLM yang terintegrasi dengan Jenkins, GitLab, dan Katalon, serta menilai efektivitasnya berdasarkan tingkat keberhasilan perbaikan, stabilitas eksekusi, dan efisiensi penggunaan sumber daya.

2.2. Landasan Teori

Beberapa konsep dasar berikut digunakan sebagai fondasi dalam merancang dan mengevaluasi sistem yang dikembangkan pada penelitian ini.

a. Self-Healing Test Automation

Self-healing test automation merujuk pada kemampuan sistem pengujian untuk menyesuaikan skrip uji secara otomatis ketika terjadi perubahan minor pada antarmuka aplikasi. Mekanisme ini biasanya berfokus pada perbaikan locator yang tidak lagi sesuai akibat perubahan atribut atau struktur DOM. Fragilitas locator merupakan salah satu penyebab utama instabilitas pada UI testing, karena perubahan kecil pada DOM dapat membuat elemen tidak lagi dapat ditemukan secara konsisten, sebagaimana ditunjukkan oleh kebutuhan teknik *similarity-based localization* untuk meningkatkan robustitas pencocokan elemen [1]. Kondisi tersebut sering memicu *false failures* dalam pipeline CI/CD sehingga menghambat kelancaran *continuous testing*. Dalam konteks DevOps, mekanisme self-healing menjadi semakin penting untuk menjaga stabilitas eksekusi regresi pada rilis yang berlangsung cepat [2].

b. Document Object Model (DOM) dan Locator

Document Object Model (DOM) adalah representasi hierarkis dari struktur halaman web yang memungkinkan identifikasi elemen UI melalui berbagai jenis locator seperti XPath, CSS selector, ID, dan atribut lainnya. Locator ini digunakan

oleh framework otomasi pengujian untuk menemukan elemen target pada saat eksekusi test. Perubahan dalam DOM dapat menyebabkan locator tidak lagi valid dan memicu kegagalan test [3]. Selain itu, stabilitas eksekusi juga dipengaruhi oleh faktor lingkungan dan infrastruktur yang dapat menghasilkan *flaky tests*, yaitu test yang tidak konsisten hasilnya meskipun tidak ada perubahan pada aplikasi [11]. Hal ini menegaskan perlunya mekanisme perbaikan locator yang lebih adaptif dan kontekstual.

c. Large Language Model (LLM)

Large Language Model (LLM) adalah model berbasis *deep learning* yang dilatih pada korpus data besar sehingga mampu memahami konteks, menalar, dan menghasilkan teks secara otomatis. Dalam domain software testing, LLM telah digunakan untuk berbagai tugas seperti *test generation*, *automated repair*, dan *maintenance support* [6]. Studi-studi awal menunjukkan bahwa LLM memiliki kemampuan untuk memahami struktur kode maupun pola perubahan sistem, sehingga dapat memberikan rekomendasi perbaikan yang relevan secara kontekstual [5], [6]. Dengan kemampuan reasoning yang luas tersebut, LLM menjadi kandidat yang potensial untuk membantu memperbaiki locator UI secara otomatis berdasarkan perubahan DOM.

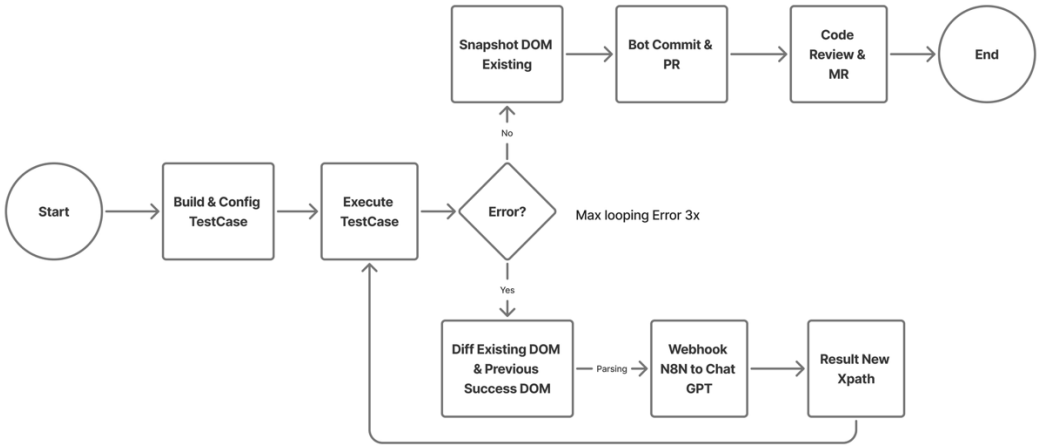
d. Verification Pipeline

Verification pipeline adalah rangkaian proses pemeriksaan yang digunakan untuk memastikan bahwa hasil keluaran model LLM valid sebelum diterapkan pada eksekusi pengujian. Keluaran LLM berpotensi mengandung kesalahan sintaks maupun ketidaksesuaian semantik, sehingga diperlukan lapisan verifikasi yang mencakup pengecekan format locator, validasi eksekusi runtime, serta pemeriksaan stabilitas terhadap kondisi DOM actual [10]. Pendekatan berbasis similarity seperti yang diperkenalkan oleh Similo [1] juga dapat mendukung proses verifikasi dengan membandingkan karakteristik elemen yang dihasilkan dengan elemen baseline. Mengingat UI testing rentan terhadap flakiness dan perubahan DOM, verification pipeline menjadi aspek penting untuk memastikan bahwa mekanisme self-healing tidak menghasilkan *false healing* yang justru memperburuk stabilitas test.

3. PERANCANGAN SISTEM

3.1. Metode yang diangkat

Metode ini mengombinasikan otomasi pengujian UI berbasis Katalon Studio dengan integrasi LLM (ChatGPT) untuk **self-healing** dalam pipeline CI/CD Jenkins. Pada dasarnya, ketika sebuah kasus uji UI gagal karena perubahan DOM, sistem akan menangkap struktur DOM terkini, membandingkannya dengan keadaan sebelumnya (DOM diff), dan menggunakan model LLM dari OpenAPI untuk menghasilkan *locator* baru yang sesuai. Algoritme ini melibatkan beberapa langkah berurutan yang dipicu otomatis dalam pipeline Jenkins, sebagaimana dirangkum berikut:



Gambar 1. Flowchart pipeline jenkins

3.1.1. Konfigurasi Awal Test Case dan Lingkungan Eksekusi

Tahap pertama dalam metode ini adalah menyiapkan lingkungan eksekusi pengujian dengan mengonfigurasi Katalon Runtime Engine (KRE) pada Jenkins serta menghubungkannya dengan repository kode otomatisasi yang disimpan di platform Git. Dalam konteks penelitian ini, repository pengujian UI berada pada GitLab sebagai bagian dari infrastruktur perusahaan, sehingga Jenkins diatur untuk melakukan *pull* kode secara otomatis dari branch yang ditentukan setiap kali pipeline berjalan.

Konfigurasi ini mencakup pengaturan kredensial akses GitLab, definisi *pipeline script*, serta integrasi KRE melalui plugin resmi Katalon atau eksekusi perintah

baris (CLI). KRE digunakan karena mendukung eksekusi test case secara headless dan konsisten di lingkungan server, yang merupakan kebutuhan utama dalam proses *continuous testing* pada CI/CD [2].

Di dalam repository GitLab, struktur proyek otomatisasi mengikuti standar Katalon Studio, yang terdiri dari *test case*, *test suite*, dan *object repository* yang berisi locator awal. Tahap ini memastikan seluruh skrip pengujian berada dalam version control sehingga setiap perubahan dapat dilacak dan pipeline selalu mengeksekusi versi terbaru dari skrip uji.

Secara metodologis, langkah ini penting untuk membangun fondasi eksekusi yang stabil dan reproducible sebelum proses self-healing dijalankan. Dengan integrasi Jenkins–GitLab–KRE, pengujian dapat dilakukan secara otomatis dan konsisten, serta setiap modifikasi pada skrip uji dapat langsung diproses melalui pipeline DevOps yang berkelanjutan.

3.1.2 Esekusi Test Suite dan Deteksi Kegagalan

Pada tahap ini, Jenkins menjalankan *Test Suite* Katalon menggunakan Katalon Runtime Engine (KRE) yang telah dikonfigurasi pada tahap sebelumnya. Eksekusi dilakukan secara headless sesuai praktik *continuous testing*, sehingga setiap perubahan pada aplikasi langsung divalidasi melalui skrip uji yang tersimpan di repository GitLab [2].

Selama proses berjalan, Jenkins memonitor hasil eksekusi dan menghasilkan *log* yang memuat status setiap langkah uji. Log ini menjadi sumber utama untuk mendeteksi kegagalan, khususnya error yang berkaitan dengan rusaknya *locator*, seperti *WebElementNotFoundException* atau kegagalan interaksi elemen akibat perubahan DOM. Deteksi error dilakukan dengan membaca pola tertentu pada log eksekusi Katalon, sehingga sistem dapat menentukan apakah pengujian gagal karena perubahan UI atau karena kesalahan lain yang tidak relevan dengan mekanisme self-healing.

Apabila tidak ditemukan error, sistem menyimpan *snapshot* DOM dari halaman yang diuji pada saat test berhasil. Snapshot ini berfungsi sebagai baseline referensi

pada eksekusi berikutnya, karena mencerminkan struktur DOM yang valid pada kondisi terakhir ketika test case berjalan dengan benar. Selain sebagai bukti keberhasilan eksekusi, snapshot ini juga digunakan untuk proses *DOM differencing* ketika kegagalan terdeteksi di masa mendatang. Dengan demikian, sistem memiliki acuan yang konsisten untuk membedakan perubahan yang bersifat normal dari perubahan yang menyebabkan kerusakan *locator*.

Tahap ini memastikan pipeline memiliki pemahaman yang akurat tentang kondisi UI pada saat test berhasil dan gagal, sehingga proses self-healing pada langkah selanjutnya dapat dilakukan berdasarkan informasi yang valid dan terverifikasi.

3.1.3. Pengambilan Snapshot DOM dan Analisis Perubahan

Ketika eksekusi test suite menghasilkan kegagalan yang diklasifikasikan sebagai *locator failure*, sistem memasuki tahap pengambilan *snapshot* DOM dan analisis perubahan struktur (DOM differencing). Tahap ini merupakan fondasi dari proses self-healing karena menyediakan konteks spesifik mengenai perubahan UI yang berpotensi menyebabkan locator tidak lagi sesuai.

Pada tahap ini, sistem pertama-tama mengambil snapshot DOM terkini dari halaman yang gagal menggunakan mekanisme *page source extraction* melalui WebDriver. Snapshot ini merepresentasikan keadaan aktual tampilan aplikasi pada saat error terjadi. Sebagai pembanding, sistem memuat snapshot DOM baseline dari eksekusi terakhir yang berhasil, yang dianggap sebagai representasi struktur UI yang valid.

Selanjutnya, kedua snapshot tersebut dianalisis melalui proses *DOM differencing* untuk mengidentifikasi perubahan-perubahan yang relevan. Analisis ini difokuskan pada atribut dan struktur yang berpotensi mempengaruhi proses lokalisasi elemen, seperti perubahan nilai *id, name, class,* serta pergeseran posisi node dalam hirarki DOM. Pendekatan ini memastikan bahwa sistem tidak hanya mengetahui bahwa sebuah locator gagal, tetapi juga memahami mengapa kegagalan itu muncul berdasarkan perubahan aktual pada antarmuka.

Untuk meningkatkan efisiensi dan akurasi pada proses self-healing, hasil differencing kemudian divalidasi dan diringkas sebelum dikirimkan ke model LLM. Validasi ini mencakup:

1. **Seleksi perubahan yang relevan**
   Sistem memprioritaskan node yang berada di sekitar locator lama atau memiliki atribut identitas signifikan, sehingga hanya perubahan penting yang dipertimbangkan.
2. **Penyaringan perubahan non-fungsional (noise filtering)**
   Perubahan yang tidak berdampak pada struktur UI (seperti perubahan style, atribut dekoratif, atau konten dinamis yang tidak relevan) dikeluarkan dari hasil diff.
3. **Penyusunan konteks yang ringkas dan efisien**
   Sistem mengekstrak potongan DOM (subtree) yang terkait dengan perubahan penting, sehingga informasi yang akan dikirimkan ke AI lebih terarah dan hemat token.

Melalui proses differencing dan validasi ini, tahap ketiga menghasilkan **konteks perubahan yang ringkas, relevan, dan terstruktur**, yang kemudian digunakan sebagai input pada tahap self-healing berbasis LLM. Pendekatan ini tidak hanya menurunkan biaya komputasi dan penggunaan token, tetapi juga meningkatkan peluang keberhasilan AI dalam menghasilkan locator yang valid dan akurat.

3.1.4 Proses Self-Healing melalui Generasi Locator Berbasis LLM

Setelah sistem memperoleh konteks perubahan DOM dari tahap sebelumnya, pipeline memasuki proses *self-healing* yang memanfaatkan Large Language Model (LLM) untuk menghasilkan locator baru yang sesuai dengan struktur UI terkini. Proses ini hanya dijalankan ketika kegagalan pengujian telah diklasifikasikan sebagai *locator failure* dan data differencing tersedia sebagai bahan analisis.

Tahap pertama dari proses ini adalah penyusunan *prompt* yang akan diberikan kepada LLM. Prompt dirancang secara terstruktur untuk memberikan konteks perubahan secara ringkas namun informatif. Informasi yang dikirimkan meliputi locator lama, potongan DOM terkait (subtree) yang diperoleh dari hasil

differencing, daftar atribut yang berubah, serta instruksi eksplisit kepada model untuk menghasilkan locator baru yang unik, stabil, dan valid. Pendekatan ini selaras dengan temuan penelitian sebelumnya, yang menunjukkan bahwa LLM dapat menghasilkan perbaikan test atau saran kode yang konsisten ketika diberikan konteks yang tepat dan terarah [3], [5], [6].

Prompt tersebut kemudian dikirimkan ke model ChatGPT melalui workflow automation n8n yang akan memanggil API dari OpenAI. Payload dikirim dalam format JSON sesuai standar API, dan respons model diproses oleh pipeline untuk mengambil kandidat locator yang dihasilkan. Locator yang dihasilkan dapat berupa XPath, CSS selector, maupun variasi lain bergantung pada instruksi yang diberikan di dalam prompt.

Sebelum digunakan dalam eksekusi ulang, setiap kandidat locator menjalani proses validasi untuk memastikan keakuratannya. Validasi ini mencakup pemeriksaan sintaksis, pengujian langsung terhadap DOM runtime untuk memastikan bahwa locator tersebut memang menemukan elemen yang tepat, serta pengecekan *unique match* untuk mencegah false healing. Apabila locator yang dihasilkan belum valid, pipeline akan melakukan percobaan ulang hingga batas maksimal tiga kali agar proses self-healing tetap efisien dan tidak mengganggu keseluruhan alur CI/CD.

Jika locator baru berhasil divalidasi, pipeline akan memperbarui konfigurasi test case dan melanjutkan eksekusi ulang pengujian dengan locator tersebut. Tahap ini merupakan inti inovasi penelitian, karena memungkinkan sistem pengujian bersifat adaptif terhadap perubahan UI tanpa memerlukan intervensi manual, sekaligus menunjukkan potensi LLM dalam mendukung proses pemeliharaan test automation di lingkungan industri.

3.1.5 Bot Commit & Pull Request (PR)

Jika proses self-healing berhasil menghasilkan locator baru yang valid dan test case kembali lulus setelah dieksekusi ulang, pipeline memasuki tahap final berupa pembaruan baseline dan integrasi perubahan ke dalam repository pengujian. Tahap ini bertujuan memastikan bahwa perbaikan yang ditemukan oleh sistem tidak hanya

menyelesaikan kegagalan pada satu eksekusi, tetapi juga menjadi bagian permanen dari test suite untuk menjaga stabilitas pengujian di masa mendatang.

Langkah pertama adalah menyimpan *snapshot* DOM terbaru sebagai baseline yang valid. DOM ini dianggap sebagai representasi paling mutahir dari struktur UI ketika test berhasil, dan akan menjadi acuan pada eksekusi selanjutnya apabila terjadi perubahan UI kembali. Dengan mengganti baseline lama, sistem dapat beradaptasi secara progresif mengikuti evolusi antarmuka aplikasi.

Selanjutnya, pipeline memperbarui skrip pengujian dengan locator hasil self-healing. Pembaruan dilakukan melalui mekanisme *automated commit* yang dijalankan oleh bot. Bot membuat branch khusus, memasukkan perubahan pada file test object atau test script, dan kemudian membuka *pull request* (PR) atau *merge request* (MR) ke repository GitLab. Langkah ini memastikan bahwa setiap perbaikan tercatat dalam version control dan dapat ditinjau oleh tim QA.

Proses PR kemudian ditinjau oleh QA engineer untuk memastikan bahwa perubahan locator sesuai dengan standar kualitas, tidak menimbulkan regresi, dan benar-benar mewakili elemen UI yang dimaksud. Setelah mendapatkan persetujuan, perubahan tersebut digabungkan ke branch utama repository sehingga test suite diperbarui secara permanen.

Tahap ini sekaligus menandai keberhasilan penuh dari mekanisme self-healing. Dengan memperbarui baseline dan mengintegrasikan perubahan locator yang benar, pipeline memastikan bahwa siklus pengujian berikutnya berjalan dengan kondisi yang lebih stabil dan akurat. Hal ini memperkuat sifat adaptif dari metode yang diusulkan, sekaligus mengurangi kebutuhan pemeliharaan manual terhadap test suite di lingkungan CI/CD.

3.2. Metode Validasi

Metode validasi dalam penelitian ini dirancang untuk menghasilkan bukti empiris yang mampu menjawab tiga rumusan masalah terkait efektivitas sistem self-healing

berbasis Large Language Model (LLM). Validasi dilakukan dengan memanfaatkan kumpulan test case yang sengaja dibuat mengalami kegagalan akibat kerusakan locator sehingga dapat merepresentasikan skenario kesalahan yang sering terjadi dalam proses pengujian antarmuka. Seluruh proses validasi dilaksanakan menggunakan dataset yang sama agar perbandingan antarpendekatan bersifat adil, konsisten, dan bebas bias.

3.2.1. Validasi Perbandingan Tingkat Keberhasilan Perbaikan Locator

Rumusan masalah pertama dievaluasi melalui perbandingan tingkat keberhasilan perbaikan locator antara pendekatan self-healing konvensional dan pendekatan berbasis LLM. Seluruh test case yang mengalami kegagalan akibat perubahan struktur atau atribut pada UI dianalisis menggunakan kedua metode tersebut. Pendekatan konvensional mengandalkan teknik rekonstruksi locator berbasis aturan, seperti pencarian atribut alternatif atau kemiripan pola DOM, sedangkan pendekatan berbasis LLM menghasilkan rekomendasi perbaikan melalui interpretasi konteks elemen dan struktur halaman.

Keberhasilan perbaikan didefinisikan sebagai kondisi ketika test case dapat kembali berinteraksi dengan elemen target secara benar dan lulus pada pengujian ulang tanpa menyebabkan interaksi salah (false fix). Definisi operasional ini diterapkan untuk memastikan bahwa perbaikan yang dihasilkan tidak hanya membuat pengujian terlihat lulus, tetapi juga valid secara semantik. Nilai keberhasilan kemudian dihitung dalam bentuk persentase perbaikan terhadap total test case yang gagal. Melalui perbandingan ini, penelitian dapat menilai sejauh mana LLM memberikan keunggulan atau keterbatasan relatif terhadap metode self-healing yang telah digunakan secara umum di industri.

3.2.2. Validasi Efisiensi Waktu Perbaikan dan Dampaknya terhadap Reliabilitas Test Suite

Rumusan masalah kedua berfokus pada evaluasi efisiensi waktu yang dihasilkan oleh penerapan LLM serta pengaruhnya terhadap reliabilitas keseluruhan test suite. Pengukuran dilakukan dengan membandingkan waktu yang diperlukan QA engineer untuk memperbaiki locator secara manual dengan waktu yang dihasilkan

oleh sistem LLM dalam memproses perbaikan pada test case yang sama. Pengukuran waktu manual memperhitungkan variasi kinerja manusia dan dilakukan pada konteks yang dikendalikan, sementara waktu pemrosesan LLM dicatat dari saat permintaan dikirim hingga rekomendasi perbaikan diterima.

Hasil perbaikan tersebut kemudian diterapkan dan dieksekusi ulang untuk melihat dampaknya terhadap tingkat kelulusan test suite. Peningkatan jumlah test case yang kembali lulus setelah perbaikan menjadi indikator naiknya reliabilitas sistem pengujian. Melalui pendekatan ini, penelitian tidak hanya menilai keuntungan waktu yang diperoleh dari otomatisasi berbasis LLM, tetapi juga menunjukkan kontribusinya terhadap stabilitas pipeline pengujian, yang merupakan aspek penting dalam praktik TestOps

3.2.3. Validasi Efektivitas LLM dalam Perbaikan Locator UI

Rumusan masalah ketiga dievaluasi dengan menilai efektivitas LLM secara independen, tanpa membandingkannya dengan metode baseline. Analisis dilakukan dengan menghitung proporsi test case gagal yang berhasil diperbaiki oleh LLM secara benar. Penilaian keberhasilan memperhatikan ketepatan interaksi elemen serta konsistensi hasil ketika perbaikan diterapkan pada berbagai jenis kerusakan locator, termasuk perubahan atribut, pergeseran struktur DOM, serta elemen dinamis yang memiliki tingkat kesulitan lebih tinggi.

Validasi ini juga mempertimbangkan potensi terjadinya false fix dan memastikan bahwa LLM tidak hanya menyelesaikan kegagalan secara permukaan, tetapi menghasilkan perbaikan yang dapat dipertanggungjawabkan secara fungsional. Dengan demikian, analisis efektivitas memberikan gambaran menyeluruh mengenai kemampuan LLM sebagai mekanisme self-healing yang berdiri sendiri, serta menunjukkan sejauh mana model dapat diandalkan untuk menanggulangi kegagalan locator dalam konteks pengujian otomatis.

DAFTAR PUSTAKA

[1] M. Nass, E. Alégroth, R. Feldt, M. Leotta, and F. Ricca, "Similarity-based Web Element Localization for Robust Test Automation," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 3, Apr. 2023, doi: 10.1145/3571855.

[2] A. R. Patel and S. Tyagi, "Enhancing software quality of CI/CD pipeline through continuous testing: a DevOps-driven maintainable hybrid automation testing framework," *International Journal of Information Technology (Singapore)*, vol. 17, no. 7, pp. 4119–4133, Sep. 2025, doi: 10.1007/s41870-025-02578-x.

[3] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software Testing With Large Language Models: Survey, Landscape, and Vision," *IEEE Transactions on Software Engineering*, vol. 50, no. 4, 2024, doi: 10.1109/TSE.2024.3368208.

[4] Y. Wang, M. Pyhäjärvi, and M. V. Mäntylä, "Test Automation Process Improvement in a DevOpsTeam: Experience Report," Apr. 2020, doi: 10.1109/ICSTW50294.2020.00057.

[5] N. Alshahwan *et al.*, "Automated Unit Test Improvement using Large Language Models at Meta," in *FSE Companion - Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, Association for Computing Machinery, Inc, Jul. 2024, pp. 185–196. doi: 10.1145/3663529.3663839.

[6] J. Liu, L. Lemner, L. Wahlgren, G. Gay, N. Mohammadiha, and J. Wennerberg, "Exploring the Integration of Large Language Models in Industrial Test Maintenance Processes," Jul. 2025, [Online]. Available: http://arxiv.org/abs/2409.06416

[7] Z. Nan, Z. Guo, K. Liu, and X. Xia, "Test Intention Guided LLM-Based Unit Test Generation," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, IEEE, Apr. 2025, pp. 1026–1038. doi: 10.1109/ICSE55347.2025.00243.

[8] Z. Wang, K. Liu, G. Li, and Z. Jin, "HITS: High-coverage LLM-based Unit Test Generation via Method Slicing," in *Proceedings - 2024 39th ACM/IEEE International Conference on Automated Software Engineering, ASE 2024*, Association for Computing Machinery, Inc, Oct. 2024, pp. 1258–1268. doi: 10.1145/3691620.3695501.

[9] M. Paasivaara and P. Kruchten, "Agile Processes in Software Engineering and Extreme Programming-Workshops." [Online]. Available: http://www.springer.com/series/7911

[10] Y. Chen and R. Jabbarvand, "Neurosymbolic Repair of Test Flakiness," 2024, doi: 10.1145/365021.

[11] D. Silva *et al.*, "The Effects of Computational Resources on Flaky Tests," *IEEE Transactions on Software Engineering*, vol. 50, no. 12, pp. 3104–3121, 2024, doi: 10.1109/TSE.2024.3462251.

[12] K. Barbosa, R. Ferreira, G. Pinto, M. D'Amorim, and B. Miranda, "Test Flakiness Across Programming Languages," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, 2023, doi: 10.1109/TSE.2022.3208864.

LAMPIRAN

Data-data pendukung, *curriculum vitae*(CV) untuk pembimbing dari luar Universitas Telkom, dsb.

IRHAM HAFIDZ MASYFI

Jalan Otista Garut – Phone 085321785300 – irhamhafidz30@gmail.com

EDUCATION

Telkom University — **Bandung, Indonesia**
Bachelor Degree of Informatics Engineering — Graduated at August 2017
[redacted]

JOB EXPERIENCES

PT Industri Telekomunikasi Indonesia — **Bandung, Indonesia**
Secretary Department – Web Develop — *June 2015 – July 2015*
Made a web application using codeigniter framework for Secretary Department to collect all document from other department using local server.

PT. Bandung Techno Park — **Bandung, Indonesia**
Business Acceleration – Web Develop — *May 2016 – June 2016*
The main project made a web application using Laravel framework. Other job, maintenance Immersa-lab web, and made a Landing Page for Immersa-lab web.

PT. Infomedia Nusantara — **South Jakarta, Indonesia**
IT CRM Development - System Analyst — *April 2018 - Now*
Design the CRM application and Omnichannel product for support the Contact Center Service. Beside that, support team to solve the troubleshoot in contact center service.

EXTRA CURRICULAR ACTIVITIES

AKSARA Telkom University Journalism — **Bandung, Indonesia**
Member of Organization — *September 2014 – June 2017*
Especially in Public Relation Department. Beside made some news about campus, make some relation between AKSARA and other Campus Journalism in Bandung.

Chief of Public Relation Dies Natalis Aksara 2016 — *January 2016 – April 2016*
Coordinate 60 member especially Public Relation member to spread information about this event to senior high school in Bandung, and other people in Indonesia.

Participant in Two Days UX Specialty Training — **Jakarta, Indonesia** — *27-28 November 2018*
Training about how to get best UX that will be applied to the system that will be develop.

PROFESSIONAL SKILLS

Soft Skills : Problem Solving, team-work, Analytical Thinking, Management
Programming Skills : SQL
Computer Skills : Microsoft Office, Photoshop, Orangescrum, Balsamiq, Figma
Language Skills : Bahasa (Native), English (Intermediate)
Interest : Analyst, Research, Documentation